Analiza Dużych Zbiorów Danych

Wykład 1: Geneza i ewolucja Big Data – od GFS do ery Chmury

dr inż. Krzysztof
Uniwersytet Warmińsko-Mazurski w Olsztynie
Wydział Matematyki i Informatyki

Plan wykładu

- **1. Geneza Big Data i era Hadoopa**
  Kryzys tradycyjnych baz RDBMS • Publikacje Google (GFS, MapReduce) • Architektura HDFS • Zasada Data Locality
- **2. Doświadczenia wdrożeniowe i ograniczenia Hadoopa**
  Koszty utrzymania klastrów on-premise • The Small Files Problem • Wąskie gardło dyskowego I/O
- **3. Architektura chmurowa i separacja zasobów**
  Pamięć obiektowa (S3, ADLS, GCS) • Storage-Compute Decoupling • Klastry efemeryczne
- **4. Współczesny ekosystem i synteza inżynierska**
  Modern Data Stack • Złote zasady architektoniczne • Dyskusja problemowa

1. Skąd wziął się problem skali na przełomie wieków?

Na przełomie lat 1999–2004 świat IT zderzył się ze ścianą:

- **Eksplozja Internetu konsumenckiego:** Wyszukiwarki musiały zaindeksować miliardy dynamicznie generowanych stron WWW.
- **Dane maszynowe (Machine Data):** Logi serwerów Apache/Nginx, kliknięcia użytkowników (*clickstream*), telemetria urządzeń sieciowych.
- **Zmiana struktury danych:** Dane przestały mieścić się w sztywnych tabelach z predefiniowanym schematem (tekst, HTML, JSON).

> **Podstawowy dylemat przełomu wieków:**
> Jak przetworzyć 20 TB danych dziennie, gdy największy dostępny dysk twardy ma 80 GB, a najszybszy serwer kosztuje 100 000 USD?

2. Rozbicie mitu marketingowego: 3V / 5V

W podręcznikach Big Data często definiuje się przez słynne **"V"**:

- **Volume** (Objętość), **Velocity** (Prędkość), **Variety** (Różnorodność) ...

A jak brzmi definicja inżynierska?

> **Big Data to punkt, w którym dane przestają mieścić się na pojedynczej maszynie lub czas ich przetwarzania przekracza dopuszczalne okno biznesowe, wymuszając przetwarzanie rozproszone.**

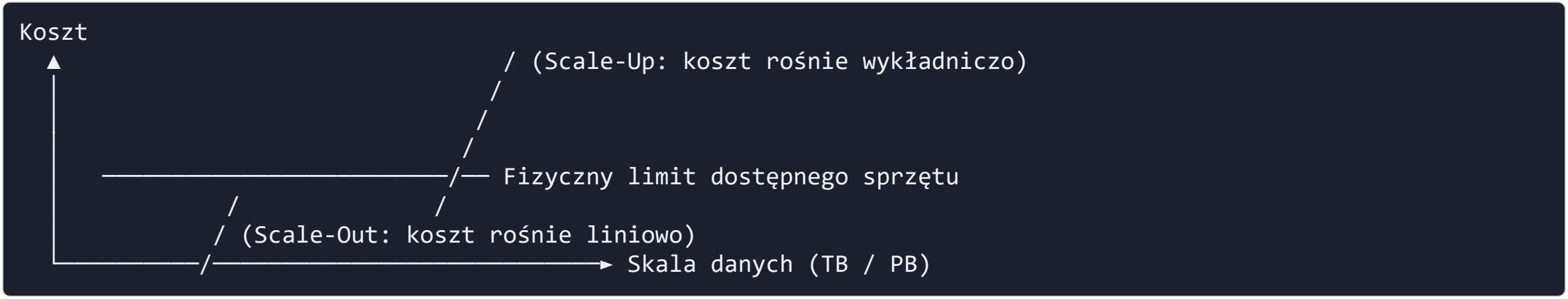
Wąskie gardło nie ma charakteru abstrakcyjnego. Jest fizyczne:

1. **Przepustowość I/O dysku:** Odczyt 1 TB z pojedynczego HDD (100 MB/s) trwa min. 2,7 godziny.
2. **Magistrala pamięci i CPU:** Ograniczenie liczby gniazd i kanałów pamięci w płycie głównej.

3. Ściana skalowania pionowego (Scale-Up)

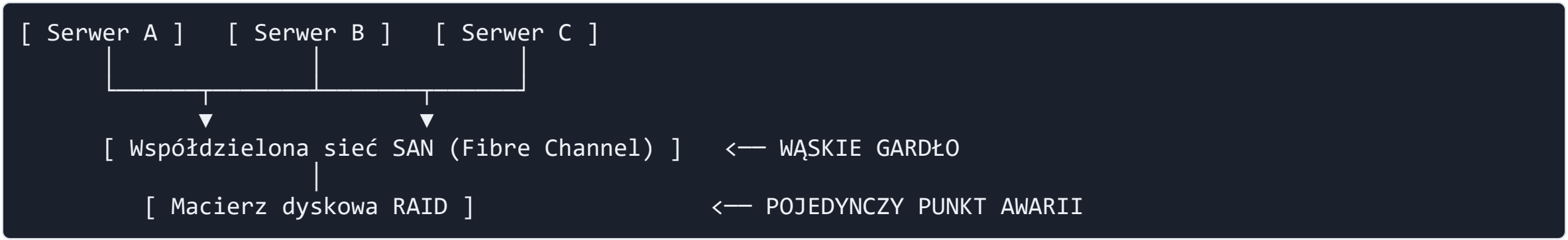
Tradycyjna odpowiedź enterprise: **„Kupmy większą maszynę!"**

- **Skalowanie pionowe (Scale-Up / Vertical Scaling):**
  - Dokładanie procesorów, RAM-u i kontrolerów do pojedynczej obudowy (np. serwery SMP / mainframe'y).
- **Ekonomia krzywej wykładniczej:**
  - Serwer o mocy $2\times$ kosztuje $4\times$ więcej.
  - Serwer o mocy $4\times$ kosztuje $16\times$ więcej.
  - Serwer o mocy $10\times$ **w ogóle fizycznie nie istnieje**.



4. Dlaczego relacyjne bazy danych (RDBMS) skapitulowały?

Klasyczny stos: Serwer bazodanowy Oracle / IBM DB2 + macierz dyskowa **SAN/NAS**.



- **Współdzielony nośnik (*Shared-Disk*):** Wszystkie węzły walczą o to samo pasmo kontrolera dyskowego.
- **Narzut transakcyjny ACID:** Blokady tabel, dwufazowy zatwierdzanie (*2-Phase Commit*) paraliżują system przy setkach transakcji na sekundę w skali rozproszonej.

5. Nowy paradygmat: Shared-Nothing i Commodity Hardware

Na początku lat 2000 inżynierowie Google podjęli radykalną decyzję:

1. **Stop drogim, wyspecjalizowanym serwerom.** Budujemy klastry ze zwykłych, tanich maszyn klasy PC (**Commodity Hardware**).
2. **Architektura Shared-Nothing:** Żaden węzeł nie współdzieli pamięci RAM ani dysku z innym węzłem. Komunikacja wyłącznie przez standardową sieć Ethernet.
3. **Awaria sprzętu to stan normalny:**
    - Jeśli klaster liczy 1 000 węzłów, każdego dnia ulega awarii kilka dysków, kości RAM lub przełączników sieciowych.
    - **Oprogramowanie musi brać odpowiedzialność za odporność na awarie**, a nie drogi sprzęt z potrójnym zasilaczem.

6. Przełom Google: Trzy publikacje, które zmieniły świat

Zamiast sprzedawać oprogramowanie, inżynierowie Google opublikowali zasady jego budowy:

- **2003: GFS (The Google File System)**
  - Sanjay Ghemawat, Howard Gobioff, Shun-Tak Leung.
  - Jak zbudować niezawodny, gigantyczny system plików z zawodnych dysków.
- **2004: MapReduce: Simplified Data Processing on Large Clusters**
  - Jeffrey Dean, Sanjay Ghemawat.
  - Model programowania rozproszonego oparty na prostych operacjach mapowania i redukcji.
- **2006: Bigtable: A Distributed Storage System for Structured Data**
  - Fay Chang, Jeffrey Dean, Sanjay Ghemawat et al.
  - Baza NoSQL dla rzadkich, wielowymiarowych tabel (fundament pod Apache HBase/Cassandra).

Źródła: S. Ghemawat et al., "The Google File System", ACM SOSP 2003; J. Dean & S. Ghemawat, "MapReduce...", USENIX OSDI 2004; F. Chang et al., "Bigtable...", USENIX OSDI 2006.

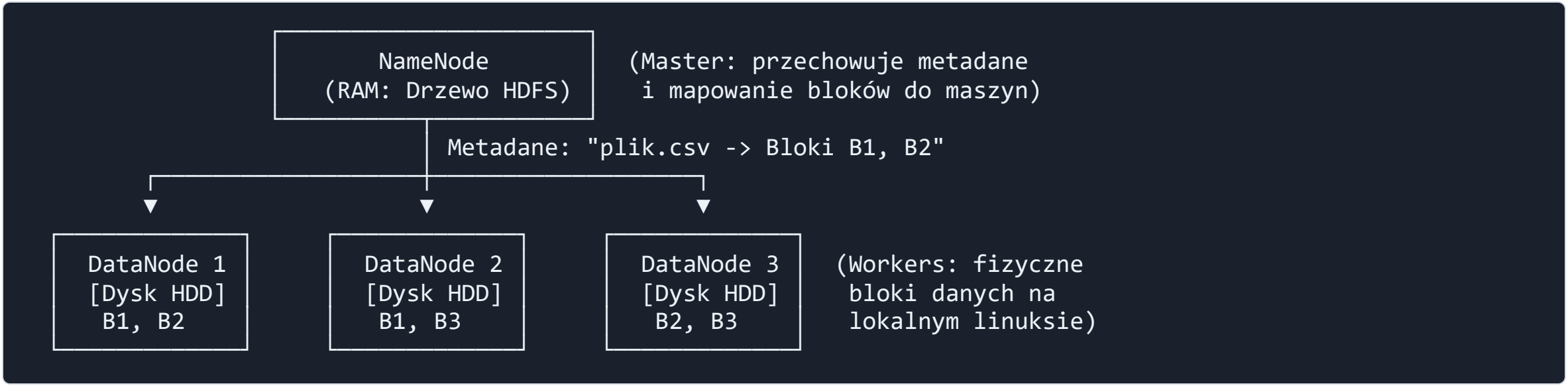
7. Narodziny Apache Hadoop (2006)

- **Doug Cutting** i **Mike Cafarella** pracowali nad projektem otwartej wyszukiwarki **Apache Nutch**.
- Po lekturze artykułów Google zaimplementowali otwarte odpowiedniki:
  - GFS $\longrightarrow$ **HDFS** (*Hadoop Distributed File System*)
  - MapReduce $\longrightarrow$ **Apache MapReduce**
- Nazwa i logo (żółty pluszowy słoń) pochodziły od ulubionej zabawki syna Douga Cuttinga.
- Projekt został wdrożony i sfinansowany przez **Yahoo!** w 2006 r. na klastrach liczących tysiące węzłów.

> Hadoop zdemokratyzował Big Data: każda firma mogła pobrać darmowy kod open-source i zbudować własne "małe Google".

8. Anatomia HDFS: Architektura Master-Worker

HDFS dzieli pliki na duże bloki (domyślnie **64 MB** lub **128 MB**) i rozprasza je po węzłach klastra.

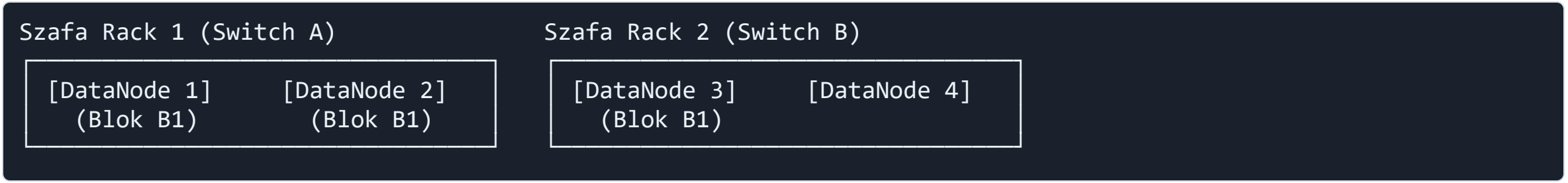


- **NameNode (Master):** Pojedynczy punkt decyzyjny. Nie przechowuje treści plików – trzyma całą strukturę katalogów i listę bloków **w pamięci RAM**.
- **DataNodes (Workers):** Przechowują surowe bloki na swoich lokalnych dyskach (ext3/ext4) i wysyłają do NameNode sygnał życia (*Heartbeat*).

9. Odporność na awarie w HDFS: Replikacja i Rack Awareness

Co się dzieje, gdy dysk lub cała szafa serwerowa ulegnie spaleniu?

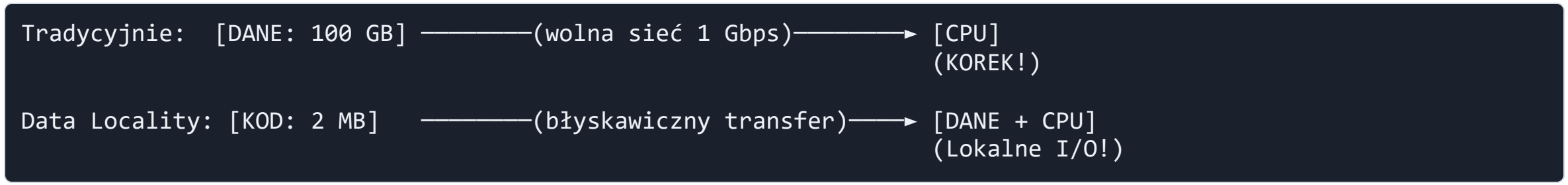
- **Domyślna replikacja $3\times$:** Każdy blok danych jest fizycznie powielany na 3 różnych maszynach.
- **Topologia świadoma szafy (*Rack Awareness*):**
  - **Kopia 1:** Lokalny węzeł, na którym uruchomiono zapis.
  - **Kopia 2:** Inny węzeł w **tej samej** szafie serwerowej (szybki transfer w ramach przełącznika top-of-rack).
  - **Kopia 3:** Węzeł w **zupełnie innej** szafie serwerowej (ochrona przed awarią zasilania szafy lub przełącznika).



10. Zasada „Data Locality”: Wyślij kod do danych!

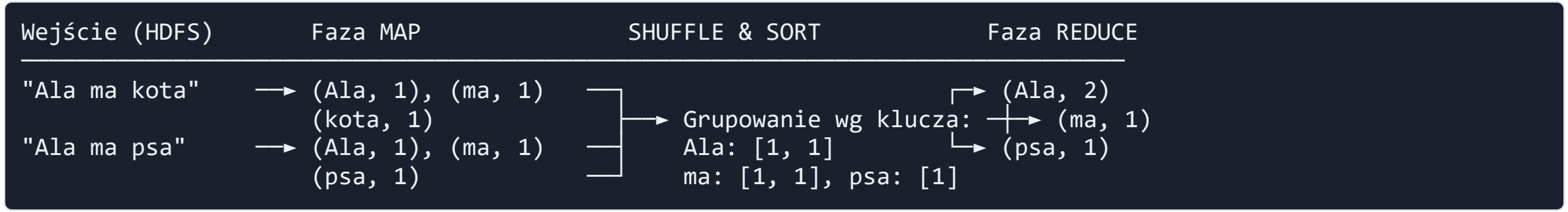
W 2005 roku sieć w serwerowni miała przepustowość **1 Gbps**, a dyski SATA odczytywały dane z prędkością **60–100 MB/s**.

- Jeśli masz **50 TB** danych, przesłanie ich przez sieć zajęłoby **ponad 110 godzin**!
- Zamiast ciągnąć petabajty danych do serwera obliczeniowego, robimy coś odwrotnego:
  - **Przesyłamy skompilowany plik programu (JAR o wadze 2 MB) bezpośrednio na maszynę, na której leży blok danych!**



11. Model MapReduce w akcji (WordCount)

Programowanie rozproszone sprowadzone do dwóch funkcji: **Map** i **Reduce**.



1. **Map (równolegle na każdym DataNode):** Przetwarza lokalny blok danych i generuje pary `(klucz, wartość)`.
2. **Shuffle & Sort (magia sieciowa):** Przesyła dane przez sieć tak, aby **wszystkie wartości dla danego klucza trafiły do jednego Reducera**.
3. **Reduce:** Agreguje wartości dla każdego unikalnego klucza i zapisuje wynik do HDFS.

12. Ewolucja: Hadoop 2.0 i YARN (2013)

W pierwszej wersji Hadoop potrafił uruchamiać **wyłącznie zadania MapReduce**.

- **YARN (Yet Another Resource Negotiator):**
  - Oddzielenie **zarządzania zasobami klastra** (pamięć RAM, rdzenie vCPU) od **modelu obliczeniowego**.
  - **ResourceManager (Master):** Przydziela kontenery obliczeniowe w klastrze.
  - **NodeManager (Worker):** Monitoruje zużycie zasobów na pojedynczym węźle.

> **Skutek:** Hadoop przestał być tylko maszynką do MapReduce. Na klastrze HDFS można było uruchamiać inne silniki: Apache Tez, Apache Storm, Apache Flink oraz **Apache Spark**.

Doświadczenia wdrożeniowe

Ograniczenia i realne koszty klastrów Hadoop on-premise

13. Studium przypadku: Radość, która zamieniła się w koszmar

Wyobraźmy sobie rok 2012 w dużej instytucji finansowej / e-commerce:

1. **Euforia (Miesiące 1–6):** Zarząd kupuje 80 serwerów, szafy rackowe, przełączniki Cisco. Instalujemy Hadoopa (Cloudera/Hortonworks). Mamy Big Data!
2. **Zderzenie (Miesiące 6–18):**
    - Zespoły analityczne narzekają: proste zapytanie SQL trwa 45 minut.
    - Klaster pada w nocy, bo padł dysk z NameNode.
    - Dział Security rwie włosy z głowy (brak Kerberosa, brak uprawnień kolumnowych).
3. **Kac finansowy (Rok 3):**
    - Klaster ma zajęte 80% dysków, ale CPU nudzi się na 10%. Żeby dołożyć dyski, trzeba... kupić kolejne serwery z drogimi procesorami!

14. Pułapka HDFS: The Small Files Problem

HDFS został zaprojektowany dla **tysięcy ogromnych plików (GB/TB)**, a nie miliardów małych.

- Każdy plik, katalog i blok w HDFS wymaga obiektu w pamięci RAM NameNode'a (~**150 bajtów**).
- **Katastrofalny scenariusz produkcyjny:**
  - System IoT generuje co sekundę plik JSON o rozmiarze **5 KB**.
  - W ciągu roku mamy 50 000 000 plików.
  - Łączny rozmiar danych: zaledwie **250 GB** (zmieściłoby się na pendrivie!).
  - **Zajętość RAM NameNode: kilkanaście gigabajtów samych metadanych! NameNode pada z OutOfMemoryError.**

```
Duży plik (1 GB):   [ 8 bloków po 128 MB ]   ──► Tylko 8 wpisów w RAM NameNode
Milion małych (1 KB): [ 1 000 000 plików ]    ──► 1 000 000 wpisów w RAM NameNode!
```

15. Pułapka MapReduce: Dysk jako kotwica spowalniająca

MapReduce był niezawodny, ale przeraźliwie powolny z powodu swojej natury:

- **Zapis na dysk po każdym etapie:**
  i. Mapper czyta z dysku HDFS.
  ii. Mapper zrzuca wynik częściowy na lokalny dysk HDD.
  iii. Faza Shuffle przesyła dane przez sieć i zapisuje do bufora dyskowego.
  iv. Reducer wykonuje obliczenia i... zapisuje wynik na dysk HDFS (z potrójną replikacją!).
- **Algorytmy iteracyjne (Machine Learning, analiza grafów):**
  - Jeśli algorytm wymaga 20 iteracji (np. PageRank, K-Means), MapReduce wykonuje **40 operacji zapisu i odczytu z fizycznych talerzy dyskowych**.

> Większość czasu zadania MapReduce upływała na **serializacji obiektów Javy, zapisie na magnetyczny dysk i odczycie**, a nie na faktycznych obliczeniach matematycznych.

16. Koszmar Capacity Planningu (Sztywne powiązanie)

W klasycznym klastrze Hadoop **dysk i procesor są uwięzione w tej samej obudowie serwera**.

```
Problem 1: Wzrost ilości danych                Problem 2: Złożone modele ML
Brakuje nam STORAGE (dysków),                  Brakuje nam COMPUTE (mocy CPU),
ale mocy CPU mamy w nadmiarze.                 ale dyski są w 80% puste.

Kupujesz: Serwery z CPU + RAM + DYSK.          Kupujesz: Serwery z CPU + RAM + DYSK.
Marnujesz: Płacisz za bezczynne CPU!           Marnujesz: Płacisz za puste dyski!
```

- **Zarządzanie szczytami (Black Friday):**
  - Klaster musi być wyskalowany pod najwyższy szczyt w roku.
  - Przez pozostałe 360 dni w roku **70% maszyn w serwerowni zużywa prąd i generuje koszty amortyzacji bezczynnie**.

Architektura chmurowa

Pamięć obiektowa i separacja Storage od Compute

17. Rewolucja sieciowa: Śmierć reguły „Data Locality”

Dlaczego w 2004 r. Google musiało liczyć lokalnie na węzłach? Bo sieć była wąskim gardłem.
Co zmieniło się w nowoczesnych centrach danych?

Parametr	Serwerownia 2004	Nowoczesne Data Center Chmury (2024+)
Przepustowość sieci	$1\ \text{Gbps}\ (\approx 125\ \text{MB/s})$	$100 - 400\ \text{Gbps}\ (\approx 12,5 - 50\ \text{GB/s})$
Topologia sieci	Tradycyjne drzewo (wąskie gardła)	*Spine-Leaf (Clos network)* – pełna przepustowość
I/O lokalnego dysku	HDD: $80\ \text{MB/s}$	NVMe SSD: $3000 - 7000\ \text{MB/s}$

Przełom inżynierski:
Dzisiaj przesłanie bloku danych przez dedykowaną magistralę sieciową w chmurze jest **szybsze niż odczytanie go ze starego dysku magnetycznego**. Dogmat kolokacji danych i procesora przestał mieć uzasadnienie fizyczne!

Źródło danych porównawczych: Cisco Global Cloud Index & specyfikacje platform chmurowych AWS/Azure.

18. Chmurowy Object Storage (S3, ADLS, GCS)

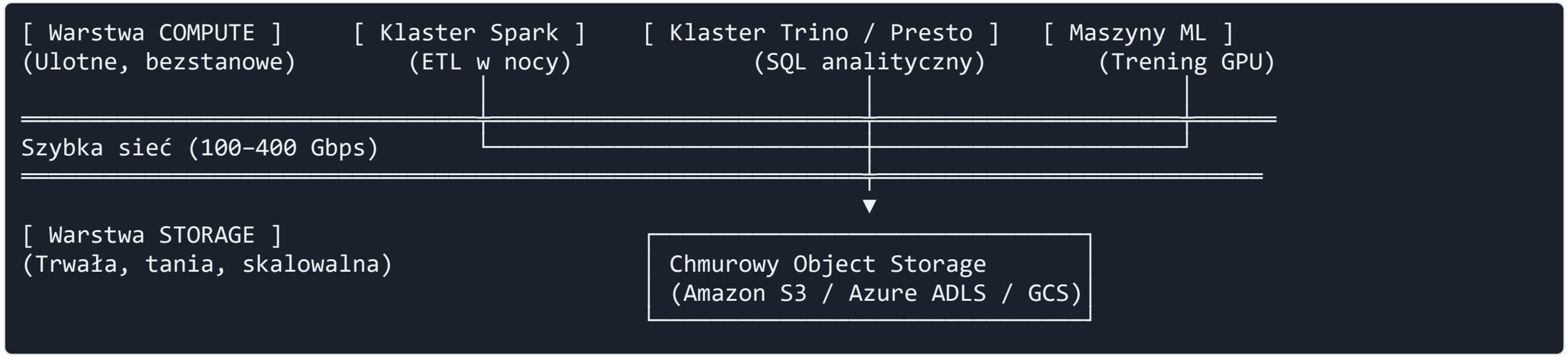
Zamiast emulować hierarchiczny system plików (z katalogami, uprawnieniami POSIX i blokami), chmura zaproponowała **pamięć obiektową**.

- **Prosty interfejs REST API:** Tylko metody `GET`, `PUT`, `DELETE`, `LIST`.
- **Brak prawdziwych katalogów:** Ścieżka `/data/2026/09/sales.parquet` to tylko jeden długi ciąg znaków (tzw. klucz obiektu).
- **Niewyobrażalna trwałość (Durability):**
  - AWS S3 gwarantuje trwałość rzędu **99.999999999% (11 dziewiątek)**.
  - Dane są automatycznie replikowane w obrębie minimum 3 niezależnych centrów danych (Availability Zones).
- **Niezrównana ekonomia:** Utrzymanie 1 TB w S3 to koszt rzędu ~20 USD miesięcznie (bez kosztów prądu, serwerowni i administratorów).

Źródło: Amazon Web Services: "Amazon S3 Storage Classes and Durability Architecture".

19. Święty Graal: Separacja Storage od Compute

To najważniejsza koncepcja architektoniczna współczesnego Big Data:



- **Storage żyje wiecznie:** Wszystkie dane organizacji lądują w jednym, bezpiecznym Data Lake.
- **Compute żyje tylko wtedy, gdy liczy:** Klastry są bezstanowe (*stateless*). Mogą być tworzone, skalowane i niszczone w kilka minut.

Koncepcja ugruntowana m.in. w: B. Dageville et al., "The Snowflake Elastic Data Warehouse", ACM SIGMOD 2016.

20. Zestawienie: HDFS on-premise vs Cloud Object Storage

Kryterium	Klastry Hadoop (HDFS on-premise)	Nowoczesny Cloud Object Storage
Zależność zasobów	Ścisłe powiązanie dysku i CPU	Całkowita niezależność skalowania
Zarządzanie	Zespół dedykowanych inżynierów	Usługa w pełni zarządzana (Serverless)
Odporność na awarie	Replikacja 3x w ramach jednej serwerowni	Geograficzna replikacja w 3 strefach (AZ)
Czas życia obliczeń	Klastry statyczne (działające 24/7/365)	Klastry ulotne (*Ephemeral*) na żądanie
Koszty bezczynności	Płacisz 100% (prąd, leasing, szafy)	Płacisz 0 USD za compute, grosze za storage
Zgodność POSIX	Częściowa (append, rename)	Brak (REST API, obiekty niezmienne)

21. Koncepcja klastrów efemerycznych (Ephemeral Compute)

W nowym modelu całkowicie zmienia się podejście do cyklu życia klastra:

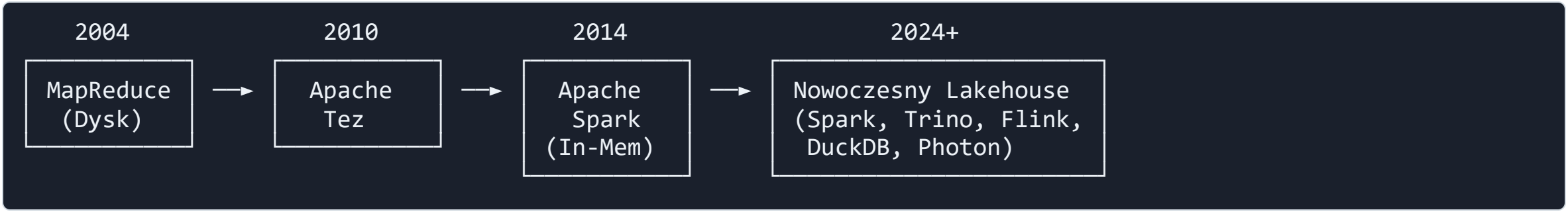
```
02:00 w nocy:    Harmonogram (np. Apache Airflow) wysyła sygnał API.
02:02:           Chmura powołuje klaster 100 węzłów (np. Databricks / AWS EMR).
02:03 - 02:25:   Klaster czyta dane z S3, przelicza raporty dobowe, zapisuje wynik.
02:26:           Wszystkie 100 maszyn zostaje BEZWZGLĘDNIE ZNISZCZONYCH.
```

Zysk biznesowy:

- Zapłaciliśmy za **23 minuty** pracy procesorów, a nie za 24 godziny!
- Zero problemów z zaśmiecaniem dysków, wyciekami pamięci czy "starzeniem się" środowiska operacyjnego.
- Każde zadanie może mieć swój własny, dedykowany klaster o idealnie dobranych parametrach (np. węzły zoptymalizowane pod RAM lub wyposażone w GPU).

22. Ewolucja silników obliczeniowych

Śmierć MapReduce otworzyła drogę do nowej generacji silników:



- **Apache Spark:** Zastąpił operacje dyskowe przetwarzaniem w pamięci RAM (**In-Memory Computing**), przyspieszając zadania od 10 do 100 razy.
- **Silniki zapytań SQL (Trino / Presto):** Umożliwiły odpytywanie gigantycznych jezior danych w czasie sekundowym (interaktywny BI).
- **Silniki strumieniowe (Apache Flink):** Przetwarzanie zdarzeń z opóźnieniem liczonym w milisekundach.

23. Co z epoki Hadoopa przetrwało, a co odeszło do lamusa?

Do muzeum technologii trafiły:

❌ Pisanie surowych zadań w Java MapReduce.
❌ Utrzymywanie własnych klastrów HDFS na fizycznym sprzęcie w serwerowni.
❌ Zasada bezwzględnej kolokacji dysku i procesora (*Data Locality*).

Na zawsze z nami zostały (DNA Big Data):

Model skalowania horyzontalnego (**Scale-Out** i architektura *Shared-Nothing*).
Filozofia akceptacji awarii pojedynczych węzłów na poziomie aplikacji.
Formaty kolumnowe wyrosłe w ekosystemie Hadoopa (**Apache Parquet, Apache ORC**).
Metastore jako katalog schematów danych (**Hive Metastore** → dzisiejszy Unity Catalog / AWS Glue).

Podsumowanie i perspektywy

Złote zasady architektoniczne i synteza ekosystemu

24. Współczesny ekosystem Big Data (Podgląd kursu)

Przez kolejne 6 wykładów będziemy zgłębiać poszczególne klocki tego nowoczesnego stosu:

```
Warstwa Orkiestracji:  [ Apache Airflow / Prefect ]                  ──► Wykład 7
                                    |
Warstwa Przetwarzania: [ Apache Spark / Databricks / Flink ]         ──► Wykłady 4, 5, 6
                                    |
Warstwa Tabelaryczna:  [ Delta Lake / Apache Iceberg ] (ACID)        ──► Wykład 3
                                    |
Formaty Plików:        [ Apache Parquet / ORC ] (Kolumnowe)          ──► Wykład 3
                                    |
Warstwa Składowania:   [ Chmurowy Object Storage: AWS S3 / ADLS ]    ──► Wykłady 1, 2
```

25. Trzy inżynierskie złote zasady Big Data

1. **Dane mają grawitację (Data Gravity):**
   - Przesyłanie petabajtów przez sieć zewnętrzną kosztuje i trwa. Trzymaj obliczenia blisko danych (w tym samym regionie chmury).
2. **Optymalizuj pod I/O, nie pod procesor:**
   - W 90% przypadków wąskim gardłem systemów Big Data nie jest brak taktowania CPU, lecz zbyt wolny odczyt i zapis danych. Dobór odpowiedniego formatu pliku to klucz do sukcesu.
3. **Nie płać za bezczynność:**
   - Jeśli klaster obliczeniowy nie przetwarza danych w tej sekundzie, powinien być wyłączony.

Termin "Data Gravity" sformułował Dave McCrory (2010).

26. Pytania sprawdzające intuicję inżynierską (Dyskusja)

Pytanie 1:
Firma posiada 50 TB logów serwerowych w plikach CSV (średnio 20 KB każdy plik). Chce załadować je do HDFS w architekturze Hadoop 1.0. Co się stanie i jak rozwiązać ten problem w chmurze?

Pytanie 2:
Przenosimy klaster obliczeniowy z serwerowni on-premise (gdzie sieć miała 1 Gbps) do AWS (gdzie węzły komunikują się przez sieć 100 Gbps). Jak ta zmiana wpływa na opłacalność stosowania formatów skompresowanych (np. Parquet + Snappy)?

Podsumowanie Wykładu 1

- Klasyczne bazy danych uległy załamaniu pod wpływem skali danych internetowych; jedyną drogą stało się **skalowanie poziome (Scale-Out)** na tanim sprzęcie.
- Google zdefiniowało fundamenty Big Data (**GFS, MapReduce**), a **Apache Hadoop** uczynił je standardem rynkowym na dekadę.
- HDFS i MapReduce borykały się z problemami: narzut I/O dysku, problem małych plików oraz sztywne powiązanie dysku z CPU.
- Rewolucja sieciowa umożliwiła **separację Storage od Compute**, wynosząc **Cloud Object Storage (S3/ADLS)** oraz **klastry efemeryczne** na pozycję standardu enterprise.

Dziękuję za uwagę!

Następny wykład (W02):

Architektury systemów Big Data i taksonomia narzędzi

Wzorce Lambda vs Kappa vs Medallion • Warstwy potoku danych • Studium przypadku end-to-end

Materiały, literatura i specyfikacje dostępne w repozytorium kursu.